

Aplicação de Conversação

FASE 1 – PROJETO DE LABORATÓRIOS DE TELECOMUNICAÇÕES E INFORMÁTICA I

Mestrado Integrado em Engenharia de Telecomunicações e Informática

Grupo 3:

- Leandro Henrique Dantas Alves A82157
- André Martins Almeida A82211
- José Eduardo da Silva Santos A82350



Índice

Índice	2
Índice de figuras e tabelas.....	3
1 Introdução	4
1 Tarefa 1.1 – Especificação do projeto.....	5
1.1 Arquitetura e funcionalidades	5
1.1.1 Camada física do sistema de comunicação (Primeira camada)	6
1.1.2 Camada de ligação de dados (Segunda camada)	6
1.1.3 Camada de aplicação (Sétima camada) e respetiva aplicação	6
1.2 Tecnologias e recursos humanos.....	7
1.3 Competências necessárias	7
1.4 Planeamento temporal	7
2 Tarefa 1.2 – Comunicação via porta série PC-Arduino-Arduino-PC.....	10
2.1 Interfaces de comunicação série	10
2.2 Ligações físicas	13
2.3 Demonstração do programa	13
2.4 Testes e resultados	16
3 Conclusão	17
4 Referências	17

Índice de figuras e tabelas

Figura 1.1 – Modelo OSI. ^[1]	5
Figura 2.1 – Níveis de tensão da interface RS-232 para o dado 0x4B.	10
Figura 2.2 – Ligação entre RS232 e UART.	11
Figura 2.3 – Formato da trama UART com 1 <i>start bit</i> , 8 bits de dados, 1 bit de paridade e 1 <i>stop bit</i>	11
Figura 2.4 – <i>Pinouts</i> do conector DS-9.....	12
Figura 2.5 – Circuito usado na comunicação série entre dois Arduinos. ^{Adaptado de [5]}	13
Figura 2.6 – Fluxograma do programa do Arduino.....	13
Figura 2.7 – Fluxograma do programa do PC (início e envio de ficheiros).	14
Figura 2.8 – Fluxograma do programa do PC (leitura de ficheiros).	15
Figura 2.9 – Forma de onda das letras ‘m’ e ‘o’ do sinal RS-232 transmitido visualizado no osciloscópio.....	16
Tabela 1.1 – Diagrama de Gantt das fases 1 e 2.....	8
Tabela 1.2 – Diagrama de Gantt das fases 3 e 4.....	9
Tabela 2.1 – Descrição de cada <i>pinout</i> do conector DS-9.	12

1 Introdução

As tarefas desta primeira fase são de especificar o projeto e de desenvolver comunicação via porta série entre um PC (*Personal Computer*) ligado a um Arduino ligado a outro Arduino ligado a outro PC.

Os objetivos a primeira tarefa incluem definir a arquitetura e as funcionalidades a serem oferecidas pelo sistema do projeto, identificar as tecnologias a serem utilizadas e fazer uma estimativa dos recursos humanos necessários para o mesmo, discutir sobre as competências necessárias incluindo as que ainda não são dominadas e fazer o planeamento temporal geral da realização dos objetivos de todas as fases, tarefas e subtarefas.

A segunda fase tem como objetivos o conhecimento das características das interfaces de comunicação série entre os módulos Arduino entre cada Arduino e o respetivo PC, conhecimento e configuração dos parâmetros de configuração da interface de comunicação série UART (*Universal Asynchronous Receiver-Transmitter*) e RS-232 (*Recommended Standard 232*) e os seus modos de funcionamento normal/transparente (*raw*), ligações físicas necessárias entre os componentes utilizados, implementação de um programa no PC para transferir ficheiros de texto e de imagem entre dois computadores ligados por cabo via porta série e implementação de um programa no Arduino para encaminhamento dos dados recebidos do PC para outro Arduino e vice-versa, utilizando uma ligação por cabo com uma interface UART ou SPI (*Serial Peripheral Interface*).

1 Tarefa 1.1 – Especificação do projeto

1.1 Arquitetura e funcionalidades

A arquitetura deste projeto consiste numa aplicação de conversação (*chat*) cujas funcionalidades são de envio e receção de mensagens de texto e de ficheiros de qualquer formato, em tempo real, entre dois computadores através dum sistema de comunicação consistido por uma rede local sem fios por RF (radiofrequência) para que haja comunicação bidirecional, fiável e simultânea entre várias estações.

O seu desenvolvimento requer a aprendizagem e utilização de funcionalidades de diferentes camadas do modelo OSI (*Open Systems Interconnection*), apresentado na Figura 1.1.

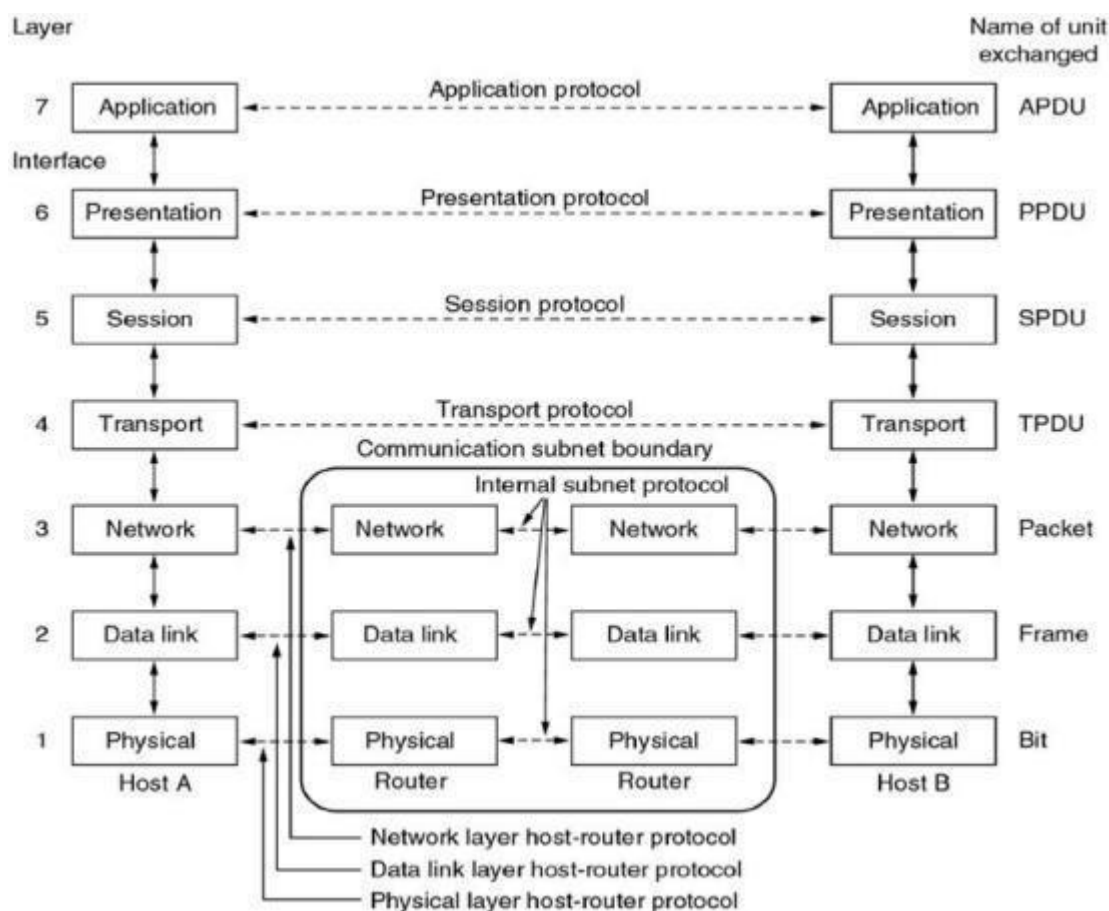


Figura 1.1 – Modelo OSI. ^[1]

Abaixo seguem-se as principais funcionalidades oferecidas por cada uma das camadas:

1.1.1 Camada física do sistema de comunicação (Primeira camada)

A camada física é responsável pela transmissão do fluxo de *raw bits* através de um canal de comunicação, fazendo com que quando um lado envia 1 bit, este é recebido no outro lado como 1 bit e não como um 0. É nesta camada que se decide que sinais elétricos devem ser usados para representar um 1 e um 0, quantos nanossegundos deve durar um bit, como é que a transmissão deve proceder simultaneamente em ambas direções, como é que a conexão inicial é estabelecida, como é que esta é desfeita quando ambos os lados estão concluídos e quantos pinos tem o conector de rede e para que serve cada um. Estas tarefas são geridas a nível elétrico, mecânico e temporal e servem para modular e desmodular o sinal, sincronizar os bits e corrigir os erros. ^[1]

1.1.2 Camada de ligação de dados (Segunda camada)

A principal tarefa da camada de ligação de dados é de transformar os dados transmitidos numa linha que se mostra livre de erros de transmissão não detetados, sendo os erros reais escondidos para que a camada de ligação não os detete. Isto é feito quando o emissor fragmenta os dados de *input* em tramas de dados e transmite as mesmas sequencialmente. Se o serviço for confiável, o recetor confirma o que recebe de cada trama através de uma trama de confirmação enviada de volta.

Um dos problemas existentes nesta camada e superiores é de como manter um transmissor rápido de abafar um recetor lento com dados. Pode ser necessário um regulador de tráfico para que o transmissor saiba se o recetor pode receber mais dados. ^[1]

1.1.3 Camada de aplicação (Sétima camada) e respetiva aplicação

O modelo OSI propõe um conjunto de 7 camadas para um sistema de comunicação, sendo a sétima camada a camada de aplicação que é contida por uma variedade de protocolos normalmente usada por utilizadores, sendo o mais comum o HTTP (*HyperText Transfer Protocol*) que serve como base da *World Wide Web* (WWW) usada nos *browsers* que quando pede uma página *Web*, este envia o endereço da página para o servidor *host* da página usando HTTP que depois é retornada.

Outros protocolos de aplicação são usados, por exemplo, na transferência de ficheiros e em correio eletrónico. ^[1]

1.2 Tecnologias e recursos humanos

As tecnologias a utilizar incluem pelo menos 2 computadores (de preferência 1 computador por cada membro do grupo), módulos Arduino (sendo recomendado 1 Arduino, como por exemplo Arduino Uno, para cada computador), *transceivers* de RF comerciais para a transmissão de dados sem fios (1 módulo rF42L01+ para cada Arduino), Arduino Software IDE (*Integrated Development Environment*), um IDE para Java (como por exemplo NetBeans ou Eclipse) e um osciloscópio digital para visualizar a forma de onda do sinal RS-232 transmitido pelo computador.

Este projeto deverá ser desenvolvido em equipa por todos os 3 membros do grupo e autonomamente por cada membro e durante as 2 sessões semanais de 3 horas cada dentro das aulas da UC (Unidade Curricular) e adicionalmente com uma dedicação mínima de 12 horas efetivas de trabalho por semana.

1.3 Competências necessárias

As competências necessárias para a realização deste projeto incluem a aprendizagem e utilização do modelo OSI, as funcionalidades associadas a cada camada, funcionalidades oferecidas pelo módulo RF através do seu *datasheet* e demais documentação relevante, redes locais sem fios (WLAN – *Wireless Local Area Network*), protocolos de aplicação específicos (por exemplo HTTP, SMTP – *Simple Mail Transfer Protocol*, FTP – *File Transfer Protocol*, etc.), programação estruturada em Java através de APIs (*Application Programming Interfaces*) para as interfaces de comunicação série (nomeadamente SPI e UART/RS-232), e trabalho em equipa e autónomo.

As competências que ainda não estão dominadas incluem as camadas do modelo OSI, o módulo RF, redes locais sem fios e protocolos de aplicação específicos, pelo que a maior parte destas competências será aprendida e aprofundada na UC de Redes de Computadores I.

1.4 Planeamento temporal

Através de dois diagramas de Gantt da Tabela 1.1 e Tabela 1.2, é demonstrado o intervalo de semanas que durou o alcance de cada objetivo de cada tarefa desta primeira fase e o planeamento temporal dos objetivos das fases seguintes.

Tabela 1.1 – Diagrama de Gantt das fases 1 e 2.

Fase	Tarefa	Objetivos	Setembro			Outubro					Novem.	
			10	17	24	1	8	15	22	29	5	12
1	1.1	Definição da arquitetura e funcionalidades a serem oferecidas pelo sistema										
		Identificação das tecnologias a serem utilizadas e estimativa dos recursos humanos necessários.										
		Discussão sobre as competências necessárias.										
		Planeamento temporal geral.										
	1.2	Conhecimento das características das interfaces de comunicação série entre os módulos Arduino e entre cada módulo Arduino e o respetivo computador pessoal.										
		Conhecimento e configuração dos parâmetros de configuração da interface de comunicação série UART/RS-232 e os seus modos de funcionamento.										
		Ligações físicas necessárias entre os componentes utilizados.										
2	2.1	Implementação de um programa no PC para a transferência de ficheiros de texto e de imagem entre dois computadores ligados por cabo via porta série.										
		Implementação de um programa no módulo Arduino para encaminhamento dos dados recebidos do PC para o outro módulo Arduino e vice-versa, utilizando uma ligação por cabo.										
		Conhecimento das características da interface de comunicação série entre cada <i>transceiver</i> de RF e o respetivo módulo Arduino.										
		Ligações físicas necessárias entre os componentes utilizados.										
	2.2	Conhecimento e compreensão do impacto no desempenho do sistema dos diferentes parâmetros configuráveis dos <i>transceivers</i> de RF.										
		Conceção e execução de testes experimentais para avaliação de desempenho do sistema.										
		Perceção das situações que se torna necessário utilizar protocolos de acesso ao meio.										
	2.2	Implementação sobre o código Arduino protocolos de controlo de acesso ao meio.										
		Avaliação do desempenho do sistema com a transmissão simultânea de dados de dois dispositivos na mesma rede.										

Tabela 1.2 – Diagrama de Gantt das fases 3 e 4.

Fase	Tarefa	Objetivos	Novembro			Dezembro					Janeir.	
			12	19	26	3	10	17	24	31	7	14
3	3.1	Percepção da noção e representação estrutural correta de trama/pacote, cabeçalho, <i>payload</i> e cauda.										
		Definição do protocolo de comunicação.										
		Definição dos tipos de trama e dos campos que estas necessitam de conter para proporcionar as funcionalidades desejadas.										
		Disponibilização das primitivas às camadas superiores através de APIs.										
		Transferência fiável de um fluxo de dados entre dois módulos Arduino sobre a ligação RF sem fios.										
4	4.1	Implementação do protocolo de nível 7.										
		Criação de um programa de conversação e transferências de ficheiros que utilize o protocolo desenvolvido.										
		Criação de uma interface gráfica para a aplicação.										
	4.2	Integração das funcionalidades desenvolvidas em todas as camadas ao longo do projeto										
		Testes experimentais finais e análise e discussão dos resultados obtidos										
		Otimização do desempenho do sistema										
		Conclusões finais referentes ao desenvolvimento do projeto como um todo.										

2 Tarefa 1.2 – Comunicação via porta série PC-Arduino-Arduino-PC

2.1 Interfaces de comunicação série

Para tal, usa-se uma conexão em série em que são enviadas e recebidas tramas a partir do parâmetro D-P-S (*data/parity/stop*), o formato típico para conexões entre *modems* e PCs é o formato “8-N-1” (8-None-1), portanto depois do *start bit*, são processados 8 bits de dados, não há bit de paridade e termina com o *stop bit* (10 bits no total). [2]

Outras alternativas incluem o “7-E-1” (7-Even-1) em que um bit de paridade par é adicionado a 7 bits de dados para um total de 8 bits entre os *start* e *stop bits*.

Pretende-se utilizar interfaces de comunicação série para comunicação com periféricos, incluindo a interface RS-232 cujos valores de tensão variam entre 3 V e 15 V quando o bit tem o valor 0 e de -15 V a -3 V quando o bit é 1, como é apresentado na Figura 2.1; assim como a interface UART de nível TTL (*Transistor-transistor logic*) cujos valores de tensão correspondentes ao bit 0 e ao bit 1 são 0 e 5 V, respetivamente. [3]

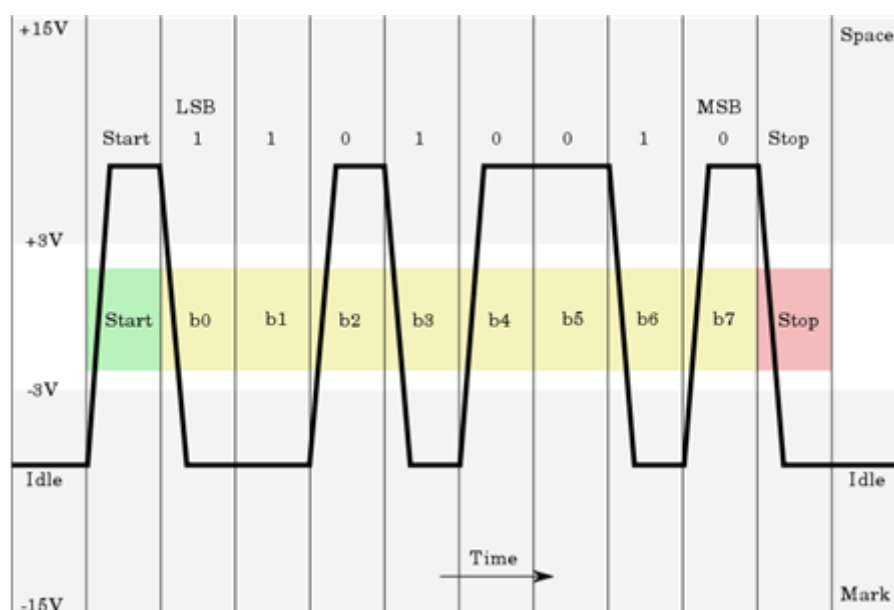


Figura 2.1 – Níveis de tensão da interface RS-232 para o dado 0x4B. [3]

Devido às diferenças de tensão e polaridade destas interfaces, as mesmas só podem ser interligadas diretamente através de um deslocador de nível. Isto pode ser feito com o circuito integrado (IC – *Integrated Circuit*) MAX232, demonstrado na figura Figura 2.2. [3]

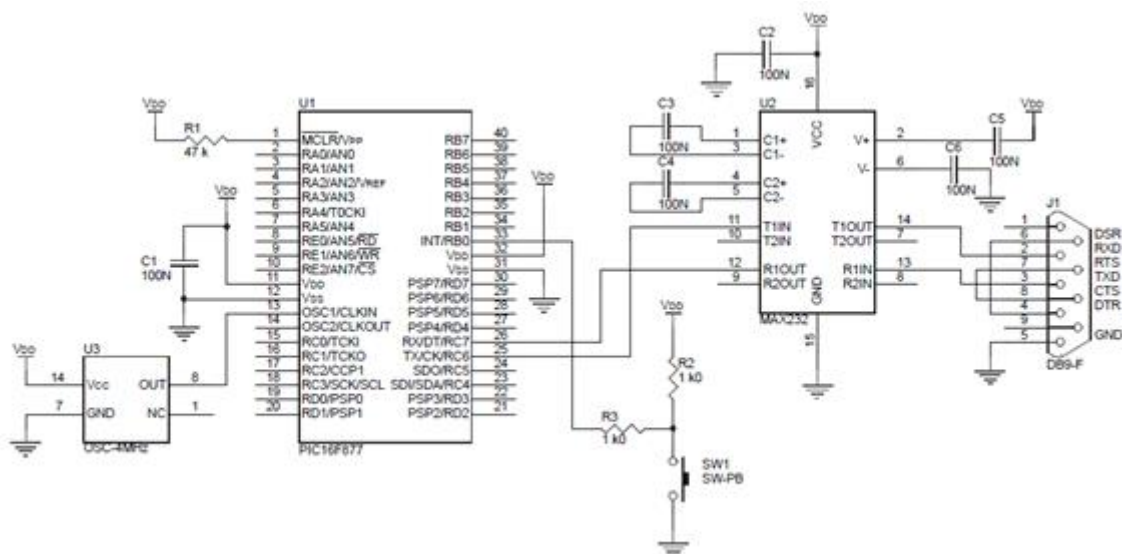


Figura 2.2 – Ligação entre RS232 e UART. [3]

A trama UART/RS-232, apresentada na Figura 2.3, é iniciada por um *start bit* que alerta o recetor que os dados estão prestes a ser enviados para que o *clock* esteja em sincronia com o *clock* do emissor, isto requer que ambos tenham o mesmo *baud rate*. O *start bit* é seguido dos bits de dados (a partir do bit mais significativo (LSB – *Least Significant Bit*) até chegar ao bit de paridade (opcional) usado para verificar a existência de erros. Para finalizar, é enviado o *stop bit* para indicar o fim da transmissão. [3] A ordem de transmissão dos bits de dados usado pelo Arduino é *little-endian* pois a ordem de armazenamento dos mesmos é de ordem crescente de significância, portanto, o LSB é o primeiro bit a ser armazenado.

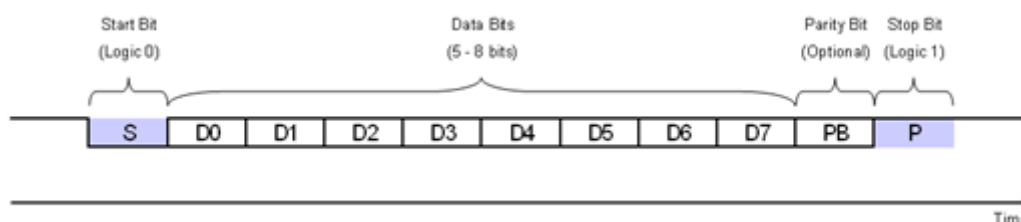


Figura 2.3 – Formato da trama UART com 1 *start bit*, 8 bits de dados, 1 bit de paridade e 1 *stop bit*. [3]

Como interface na conexão entre UART e USB (*Universal Serial Bus*), é normalmente utilizado um circuito integrado FT232. [4]

Na comunicação RS-232, é geralmente usado o conector DB-9 cujo *pinout* está representado na Figura 2.4 e as suas descrições na Tabela 2.1. Este conector não é necessário para o projeto.

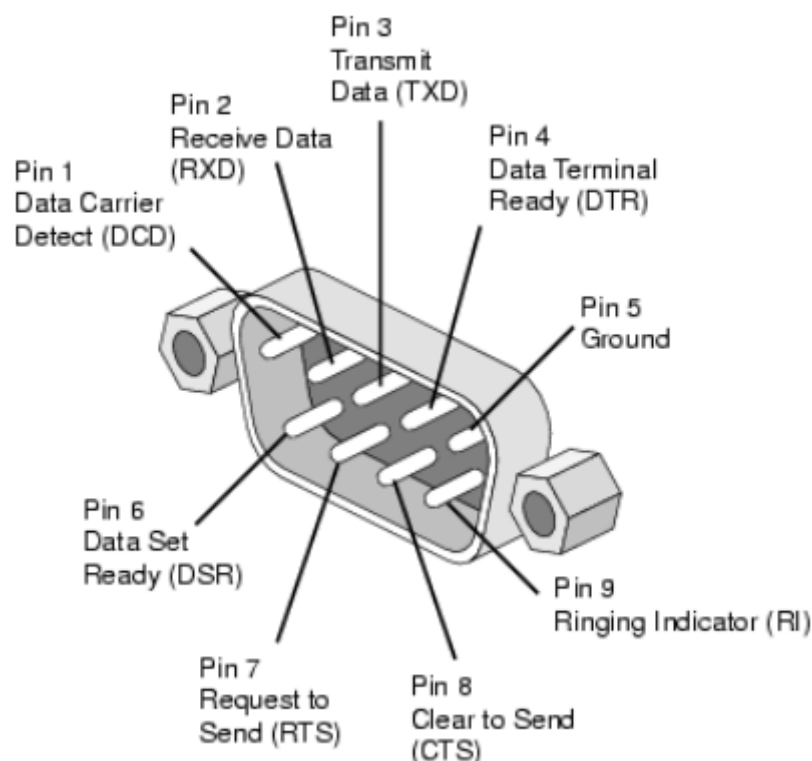


Figura 2.4 – Pinouts do conector DS-9. ^[3]

Tabela 2.1 – Descrição de cada *pinout* do conector DS-9. ^[3]

Pino	Descrição
1	Afirmação do DCE (<i>Data Communications Equipment</i>) quando é estabelecida uma conexão com o equipamento remoto
2	Entrada de dados de série
3	Saída de dados de série
4	Afirmação do DTE (<i>Data Terminal Equipment</i>) para indicar que está pronto para se conectar
5	Terra
6	Afirmação do DCE para indicar que o DCE está ligado e pronto para receber comandos ou dados para transmissão a partir do DTE
7	Linha que informa o DCE (<i>modem</i>) que o DTE (PC) está pronto a trocar dados
8	Linha que indica que o DCE está pronto a trocar dados
9	Afirmado pelo DCE quando é detetado um sinal de toque de uma linha telefónica

2.2 Ligações físicas

As ligações físicas necessárias para a comunicação série é feita através de dois PCs ligados a um Arduino cada um, que por sua vez estão ligados entre si através dos pinos ~10 ↔ ~11, ~11 ↔ ~10 e GND (Ground) ↔ GND, como é demonstrado no esquema da Figura 2.5.

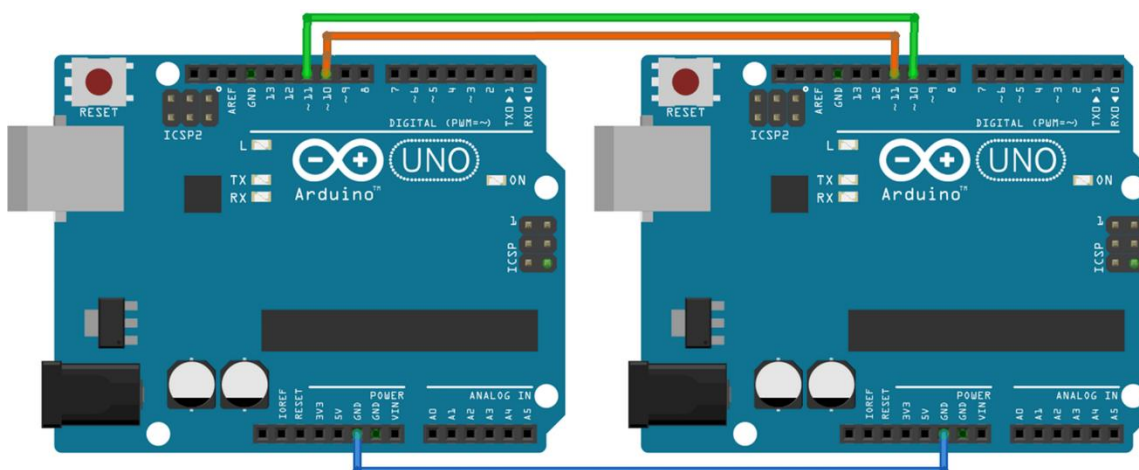


Figura 2.5 – Circuito usado na comunicação série entre dois Arduinos. Adaptado de [5]

2.3 Demonstração do programa

Foram desenvolvidos dois programas, um no PC para a transferência de ficheiros de texto e de imagem entre dois computadores e outro no módulo Arduino para encaminhar os dados recebidos do PC para o outro módulo Arduino e vice-versa, utilizando uma ligação por cabo e as ligações físicas demonstradas na secção anterior.

Estes programas estão demonstrados nos fluxogramas da Figura 2.6, Figura 2.7 e Figura 2.8.

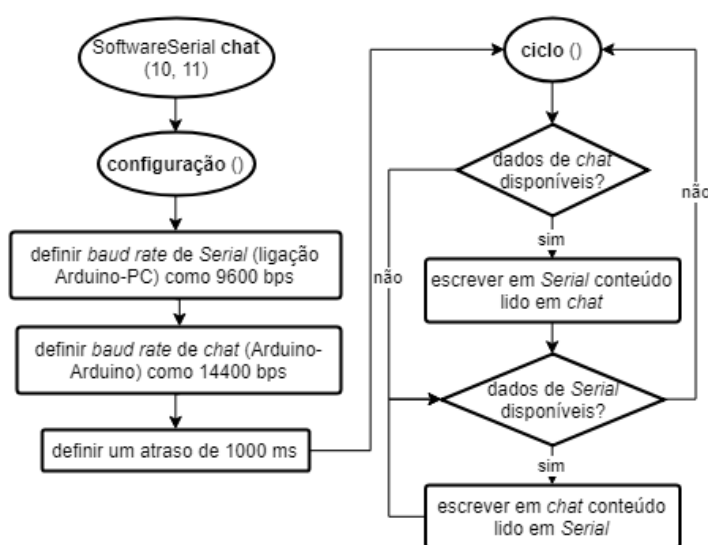


Figura 2.6 – Fluxograma do programa do Arduino.

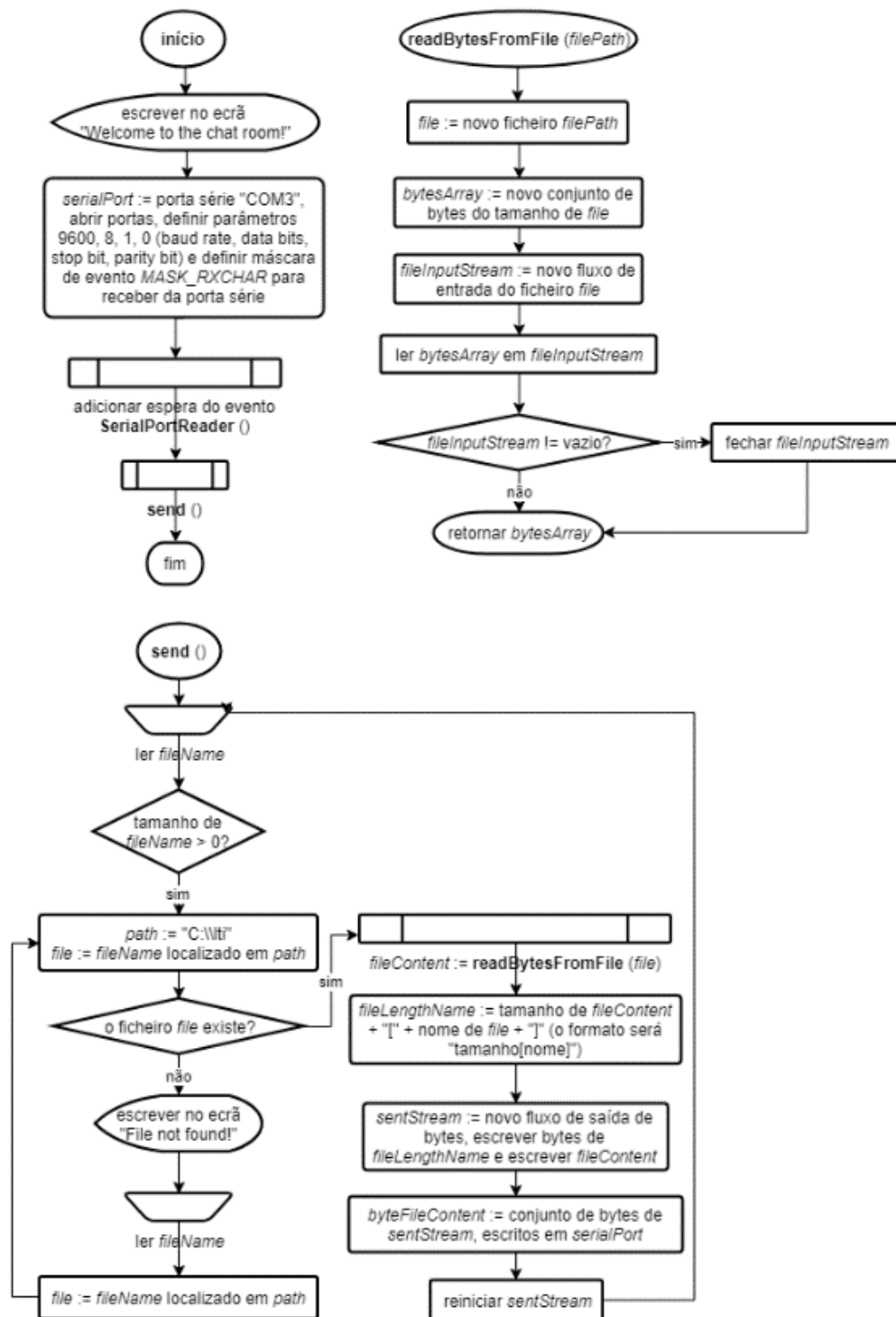


Figura 2.7 – Fluxograma do programa do PC (início e envio de ficheiros).

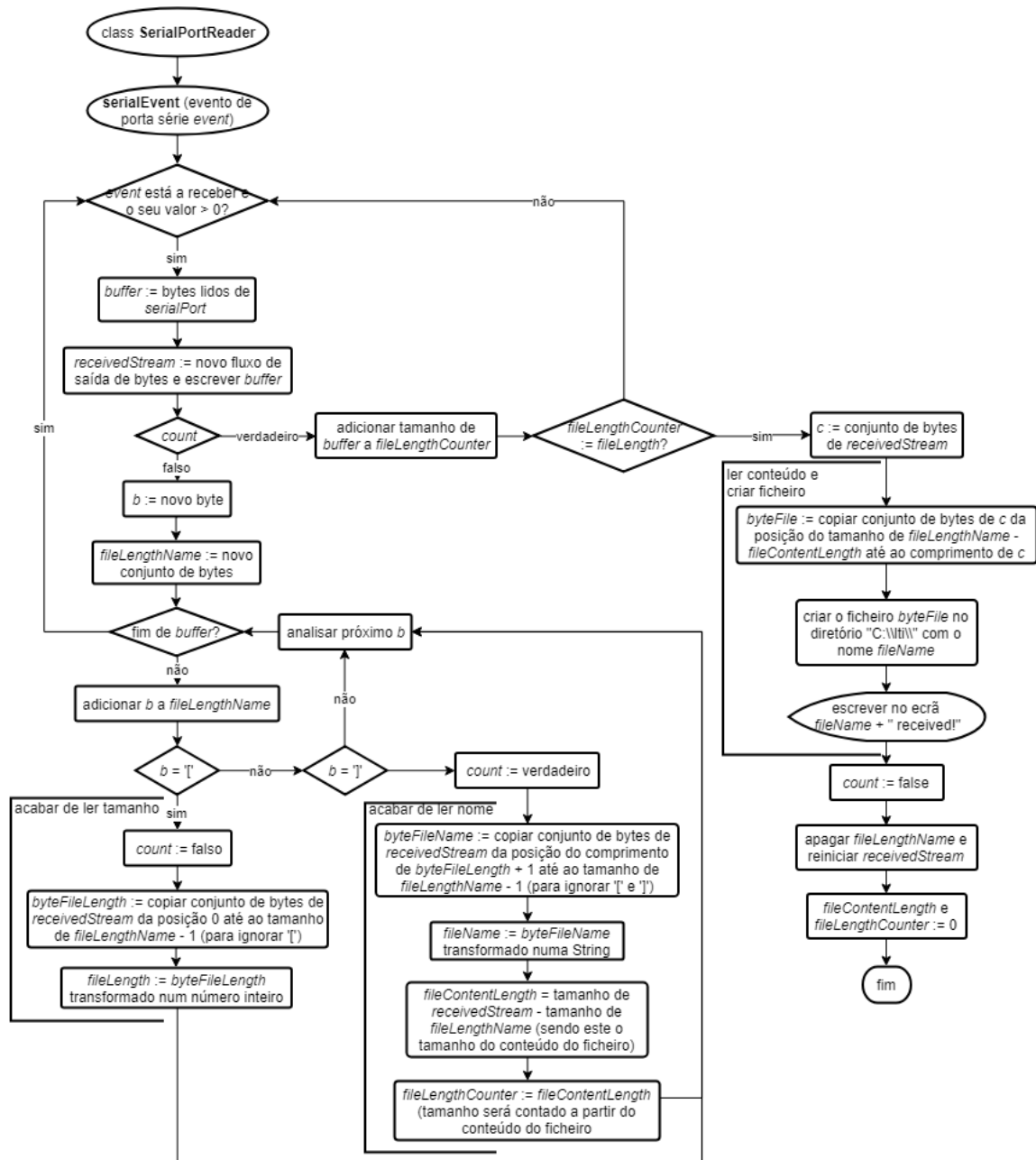


Figura 2.8 – Fluxograma do programa do PC (leitura de ficheiros).

2.4 Testes e resultados

Através de um osciloscópio digital, visualizou-se a forma de onda do sinal RS-232 transmitido pelo PC.

No exemplo da Figura 2.9, foi enviado um excerto de texto com a sequência de letras 'm' e 'o' que se convertem em 01101101 e 01101111, respetivamente em numeração binária.

Como foi exemplificado na Figura 2.1, o *start bit* é lido com nível de tensão alto e inicia a leitura de dados por ordem crescente de significância, começando pelo LSB. Os bits que têm valor 1 têm tensão baixa e os que têm valor 0 têm tensão alta. Estes dados são concluídos pelo *stop bit* de valor de tensão baixo.

Usando uma *baud rate* de 14400 bps, o tempo do bit (T_b) pode ser calculado da seguinte forma:

$$T_b = \frac{1}{R} = \frac{1}{14400} \approx 69,444 \mu s$$

Através da Figura 2.9, pode-se verificar que o valor experimental do tempo do bit é de aproximadamente 70 μs , o que condiz com o valor teórico.

Depois da trama ser lida, ocorre um intervalo de tempo (*gap*) até à leitura da próxima trama, experimentalmente pode-se verificar que este intervalo toma o valor aproximado de 340 μs .

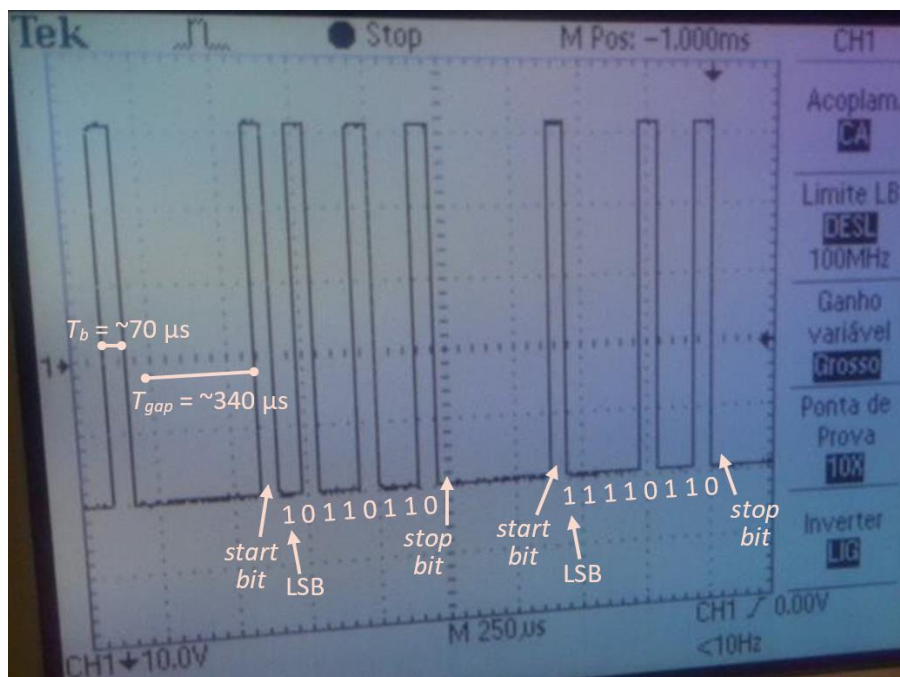


Figura 2.9 – Forma de onda das letras 'm' e 'o' do sinal RS-232 transmitido visualizado no osciloscópio.

3 Conclusão

Esta fase ajudou-nos a atingir os objetivos propostos que incluem a arquitetura, funcionalidades, tecnologias e competências necessárias para todo o projeto, assim como conseguimos transferir ficheiros de todo o tipo de extensão entre PCs através de Arduinos usando a interface UART/RS-232.

Este trabalho foi árduo pois estávamos com alguns problemas cuja origem era desconhecida durante mais de uma semana até que se descobriu que se tratava de algo simples que se complicou devido à nossa distração, mas no final tudo se resolveu e achamos que concluímos esta fase com sucesso.

4 Referências

- [1] A. S. Tanenbaum, "Computer Networks", Prentice Hall, 2003.
- [2] ModemHelp.Net FAQs - What does 8-N-1 mean?.
<http://www.modemhelp.net/faqs/8n1.shtml>
27/09/2018.
- [3] UART – Universal Asynchronous Receiver and Transmitter – Tutorial by Cytron.
<https://tutorial.cytron.io/2012/02/16/uart-universal-asynchronous-receiver-and-transmitter/>
02/10/2018.
- [4] FT232R.
<https://www.ftdichip.com/Products/ICs/FT232R.htm>
02/10/2018.
- [5] Serial Communication Between Two Arduino Boards – Arduino Project Hub.
<https://create.arduino.cc/projecthub/harshmangukiya/serial-communication-between-two-arduino-boards-d423e9>
17/09/2018.